Helix Constitution

Helix Constitution

Field	Value
Revision	1
Created	2026-05-14
Last modified	2026-05-19
Status	active
Status summary	—
Issues	none
Issues summary	—
Fixed	none
Fixed summary	—
Continuation	—

> The universal engineering constitution shared by every project that includes this repository as a Git submodule.

What this is

This repository contains the **universal, project-agnostic** rules, constraints, and conventions that every Helix project (and any other project that opts in) inherits automatically. Project-specific rules live in the consuming project's own `Constitution.md` / `CLAUDE.md` / `AGENTS.md` and **extend** what is in here.

The model is three-layer (top-to-bottom evaluation, project overrides universal when conflicts are explicit):

1. **Base layer (this submodule)** — universal rules: anti-bluff covenant, data safety, host safety, test coverage, commit/push discipline, credentials handling, documentation discipline.
2. **Project layer** — project root `Constitution.md` / `CLAUDE.md` / `AGENTS.md`: domain-specific rules, hardware-specific constraints, service-specific configuration.
3. **Subdirectory layer (optional)** — `<subdir>/CLAUDE.md` for module-local overrides. CLI agents (Claude Code et al.) merge these by walking up from the working directory.

How to consume

1. Add the submodule

In each consuming project:

```bash
git submodule add git@github.com:HelixDevelopment/HelixConstitution.git constitution

# Pin to a tag for reproducibility (recommended)
cd constitution
git checkout v1.0.0         # whatever the current stable tag is
cd ..
git add constitution
git commit -m "chore: add Helix Constitution submodule pinned to v1.0.0"
```

When teammates clone your project:

```bash
git clone --recurse-submodules <your-project-url>
# or if already cloned:
git submodule update --init --recursive
```

2. Wire the inheritance

Add a clearly-marked pointer at the top of your project's root `CLAUDE.md`:

```markdown
## INHERITED FROM constitution/CLAUDE.md

All rules in `constitution/CLAUDE.md` (and the `constitution/Constitution.md`
it references) apply unconditionally. Project-specific rules below
extend them.

@constitution/CLAUDE.md
```

Same for `AGENTS.md`:

```markdown
> Base agent rules: `constitution/AGENTS.md` — READ IT FIRST.
> The base file is authoritative for any topic not covered here.
```

And for your project Constitution:

```markdown
This constitution **extends** the Helix Universal Constitution at
`constitution/Constitution.md`. All clauses there apply unless
explicitly overridden below with an explicit `Override §X.Y`
section.
```

3. Set up the multi-upstream push

From the **constitution submodule's** root directory:

```
cd constitution
bash install_upstreams.sh
```

This reads every `Upstreams/*.sh` declaration and configures matching git remotes locally, so a single `git push` reaches every provider (GitHub, GitLab, GitFlic, GitVerse).

Alternatively, if your toolkit ships the system-wide `install_upstreams` command (Project-Toolkit `Upstreamable/`), you can invoke that instead — it reads the same `Upstreams/` directory.

4. Verify inheritance with an automated test

Every consuming project should ship a test that verifies the constitution submodule is present, at the expected pinned revision, and that the project's CLAUDE.md / AGENTS.md / Constitution all reference the submodule. See the ATMOSphere project's `test_constitution_inheritance.sh` for a reference implementation.

Contents

File	Purpose
`Constitution.md`	**The canonical universal constitution.** All clauses are project-agnostic.
`CLAUDE.md`	Universal CLAUDE.md for Claude Code agents. Imports `Constitution.md` by reference.
`AGENTS.md`	Universal AGENTS.md for every other CLI agent (Codex, Cursor, Aider, etc.).
`LICENSE`	License governing the constitution text itself.
`Upstreams/`	One `.sh` declaration file per upstream Git remote.
`install_upstreams.sh`	Configures every declared upstream as a local git remote.
`find_constitution.sh`	Helper that locates this submodule from arbitrary nested depth.
`meta_test_inheritance.sh`	Sentinel-based meta-test that verifies the inheritance gate catches a deletion of the §11.4 anchor.
`templates/`	Templates for project-specific Constitution / CLAUDE / AGENTS files.

Multi-upstream topology

This repository is hosted on four providers, with GitHub as the primary:

Remote	URL
`github` (primary)	`git@github.com:HelixDevelopment/HelixConstitution.git`
`gitlab`	`git@gitlab.com:helixdevelopment1/helixconstitution.git`
`gitflic`	`git@gitflic.ru:helixdevelopment/helixconstitution.git`
`gitverse`	`git@gitverse.ru:helixdevelopment/HelixConstitution.git`

Every commit MUST be pushed to ALL four. The `origin` remote is configured with multiple push URLs so `git push origin <branch>` fans out to every provider in one command.

Recursive inheritance

Between the consuming project's root and any submodule that uses this constitution there may be N intermediate levels. The `find_constitution.sh` helper walks up parents until it locates the constitution submodule, so nested submodules can source the helper without knowing their own depth.

Versioning

Tag the constitution at semantically meaningful checkpoints (`v1.0.0`, `v1.1.0`, `v2.0.0`). Consuming projects pin to a specific tag and upgrade deliberately. Major version bumps mean breaking changes to the universal rules; minor bumps mean additions; patches mean clarifications.

Contributing

Universal status has to be **earned**, not assumed. Project-specific content does NOT belong here. When in doubt, classify a rule as project-specific and keep it in the consuming project's own Constitution.

A rule is **universal** only if ALL are true:

1. It would be true for at least 3 unrelated projects across different domains.
2. It does not reference a specific service / schema / port / topic / table / vendor / hardware unique to one project.
3. It encodes a *policy* or *principle*, not a *configuration value*.
4. Removing it from any consuming project would be just as wrong as removing it from any other.

A rule is **project-specific** if any are true:

- Mentions a specific service, package, topic, table, ML model, protocol version, hardware, or vendor.

- Encodes a numeric threshold tuned for one codebase (coverage %, latency SLOs, retry counts).
- Describes a workaround for a bug or quirk in a dependency this project uses.

License

See `LICENSE`.